Real-world (make it feel alive with streaming) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and what each chunk carries
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and what each chunk carries

With `stream=True`, `client.chat.completions.create(...)` returns an iterable of "chunk" objects instead of one finished response object. Each chunk carries a small piece of the answer at `chunk.choices[0].delta.content` — and this can be `None` (for chunks that don't carry new text, like the very first one), so you need to guard against printing `None`.

`print(text, end="")` — the `end=""` argument replaces the default trailing newline with nothing, so consecutive `print()` calls flow together on the same line, which is what makes streamed output look like it's "typing." Also handle the `None` case cleanly: `piece = chunk.choices[0].delta.content` can be `None`, so check `if piece:` before printing, or use `piece or ""` so you're never trying to print `None` itself.

If you also want to build up the full reply text (useful if you need to save it into conversation history afterward, like in the previous exercise), start with an empty string before the loop and add each piece to it as it arrives.

Hint 2 — The plan, and almost the whole thing

```
function stream_reply(history) -> str:
    stream = call the API with messages=history, stream=True
    full_reply = ""
    for each chunk:
        piece = chunk's delta content
        if piece exists:
            print it, no newline
            add it to full_reply
```

```
    print a final newline
    return full_reply
```

```
# streaming_practice.py
def stream_reply(history: list) -> str:
    stream = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
        stream=True,
    )

    full_reply = ""
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="")
            full_reply = full_reply + piece

    print()
    return full_reply
```

`full_reply` accumulates the whole answer as it streams by, so once the loop finishes you have the complete text — useful if you then need to append it to `history` the way the previous exercise did. Write and run this version, then compare against the Solution.

Full solution: Show me the solution